

PROG3271 – Open Source Web Programming

Milestone 2

Prepared by:

David Jaiyeola

Stryfe Vidmar

Revienne Galang

Instructor:

Angelina Ziesemer

Project Description

Sproutly is designed with one goal in mind: to help people take better care of their plants. To do this, our users need top-notch plant care, and by using our app, they can more easily and effectively reach their goals. We allow users to keep track of their houseplants by providing all the details they need and setting up their own plant care reminders. We also have a vast catalog of plants that users can search through to learn or identify certain types of plants and their requirements.

Database Design

Users - stores user account information for authentication and account management

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique user identifier
username	VARCHAR(50)	user's username
email	VARCHAR(100)	User's email address
password_hash	VARCHAR(255)	Encrypted password
created_at	TIMESTAMP	Account creation date

Relationships

- One user can own many UserPlants(1:M)
- One user can have many favorites(1:M)

Plant Catalog - stores information about plant species available for browsing and identification

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique plant identifier
common_name	VARCHAR(100)	Common plant name
scientific_name	VARCHAR(100)	Scientific plant name
description	TEXT	Plant description
watering_guide	TEXT	Watering instruction
sunlight_req	VARCHAR(50)	Sunlight requirement
image_url	VARCHAR(255)	Plant image URL

Relationships

- ⌘ One PlantCatalog record can be linked to many UserPlants (1:M)
- ⌘ One PlantCatalog record can appear in many Favorites (1:M)
- ⌘ One PlantCatalog record can appear in many StoreInventory records (1:M)

User Plants - Stores plants Owned by users and supports CRUD operations.

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique plant instance identifier
user_id	INT(FK)	Reference users
catalog_id	INT(FK)	Reference PlantCatalog
custom name	VARCHAR(100)	User defined plant name
location	ENUM(LR, Ki, BR, B)	Plant location
last watered	DATE	Last watering date
date added	TIMESTAMP	Date plant was added

Relationships

- ∉ Many UserPlants belong to one User (M:1)
- ∉ Many UserPlants may reference one PlantCatalog record (M:1)
- ∉ One UserPlant can have many Reminders (1:M)

Reminders - stores care reminders for user plants

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique reminder identifier
user_plant_id	INT(FK)	Reference userPlants
reminder_type	ENUM	type of reminder
frequency_days	INT	Reminder frequency
next_due_date	DATE	Next scheduled reminder
is_completed	BOOLEAN	Completion status

Relationships

∉ Many Reminders belong to one UserPlant (M:1)

Favorites - stores plants that user has marked as favorite

Fields

Field	Datatype	Description
user_id	INT(PK, FK)	Reference Users
catalog_id	INT(PK, FK)	Reference PlantCatalog

Relationships

- ∉ Many Favorites belong to one User (M:1)
- ∉ Many Favorites belong to one PlantCatalog record (M:1)
- ∉ Creates a Many-to-Many relationship between Users and PlantCatalog

Stores - stores information about local plant stores

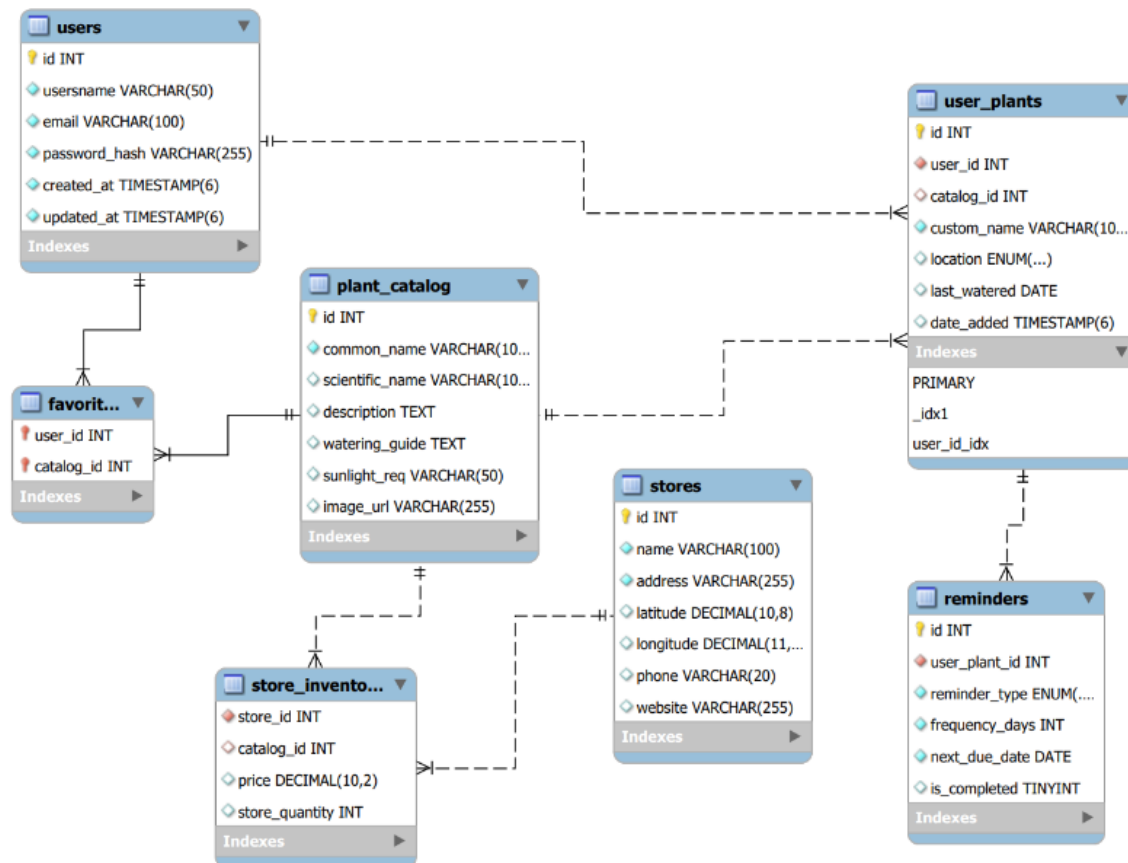
Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique store identifier
name	VARCHAR(100)	store name
address	VARCHAR(255)	store address
latitude	DECIMAL(10, 8)	Geographic latitude
longitude	DECIMAL(11, 8)	Geographic longitude
phone	VARCHAR(20)	Contact number
website	VARCHAR(255)	store website URL

Relationships

⌘ One Store can have many StoreInventory records (1:M)

ERD/Database Diagram



Frontend Wireframes

Design Philosophy – our design focuses on simplicity, intuitive navigation, and clean layout

Login screen

The wireframe shows a login screen for 'Sproutly'. At the top, there is a light green square logo placeholder, the brand name 'Sproutly', and a placeholder line of text 'Lorem ipsum dolor sit amet consectetur.'. The main content is a white card with a light gray border. Inside the card, the heading 'Welcome back' is followed by two input fields: 'Email Address' (containing 'example_user@gmail.com') and 'Password' (masked with dots and featuring a toggle icon). Below these are a 'Remember me' checkbox and a 'Forgot password?' link. A dark green 'Sign In' button with a right arrow is positioned below the password field. A separator line with the text 'OR CONTINUE WITH' is followed by two buttons for 'Google' and 'Apple'. At the bottom, a link for 'New user? Click here to sign up.' is provided.

Welcome back

Email Address
example_user@gmail.com

Password
••••••••

☐ Remember me [Forgot password?](#)

Sign In →

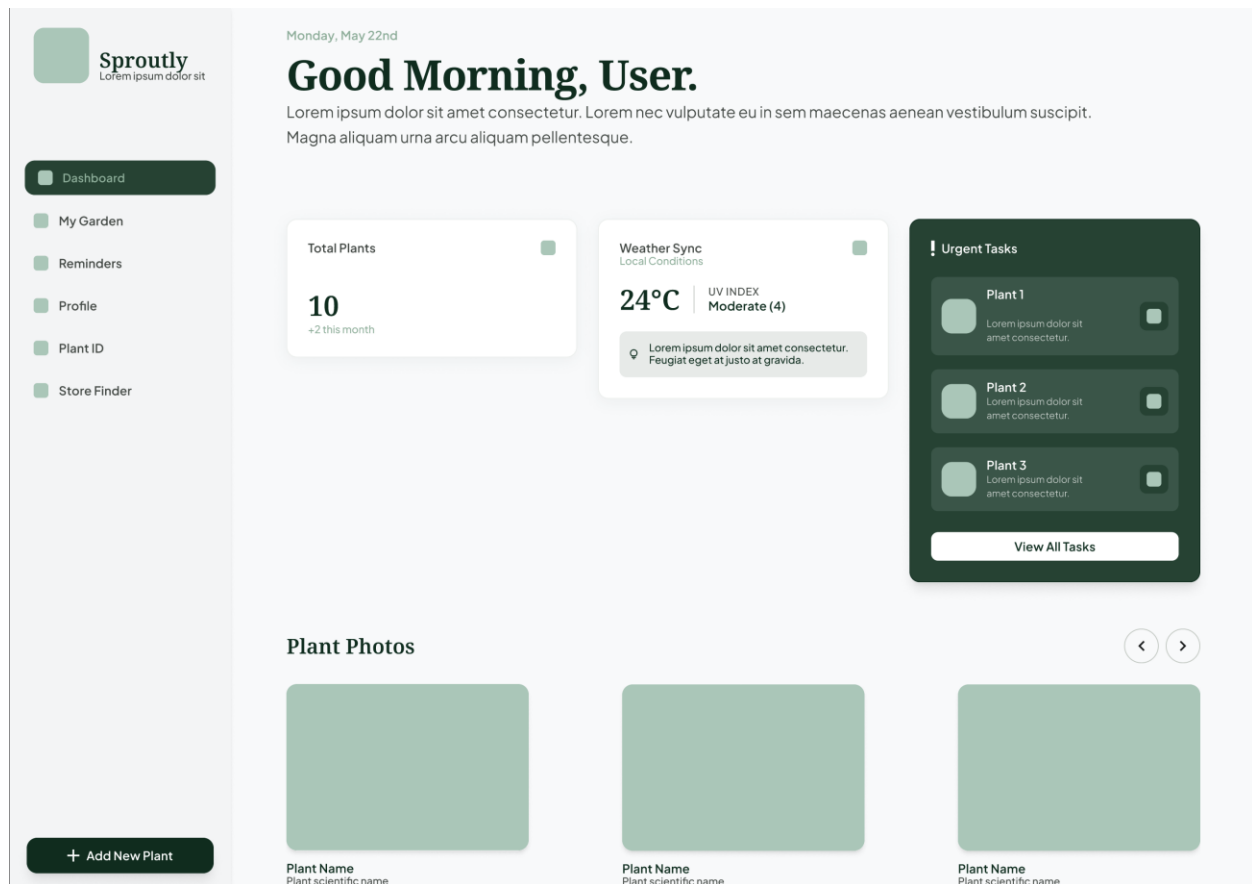
OR CONTINUE WITH

Google Apple

New user? Click [here](#) to sign up.

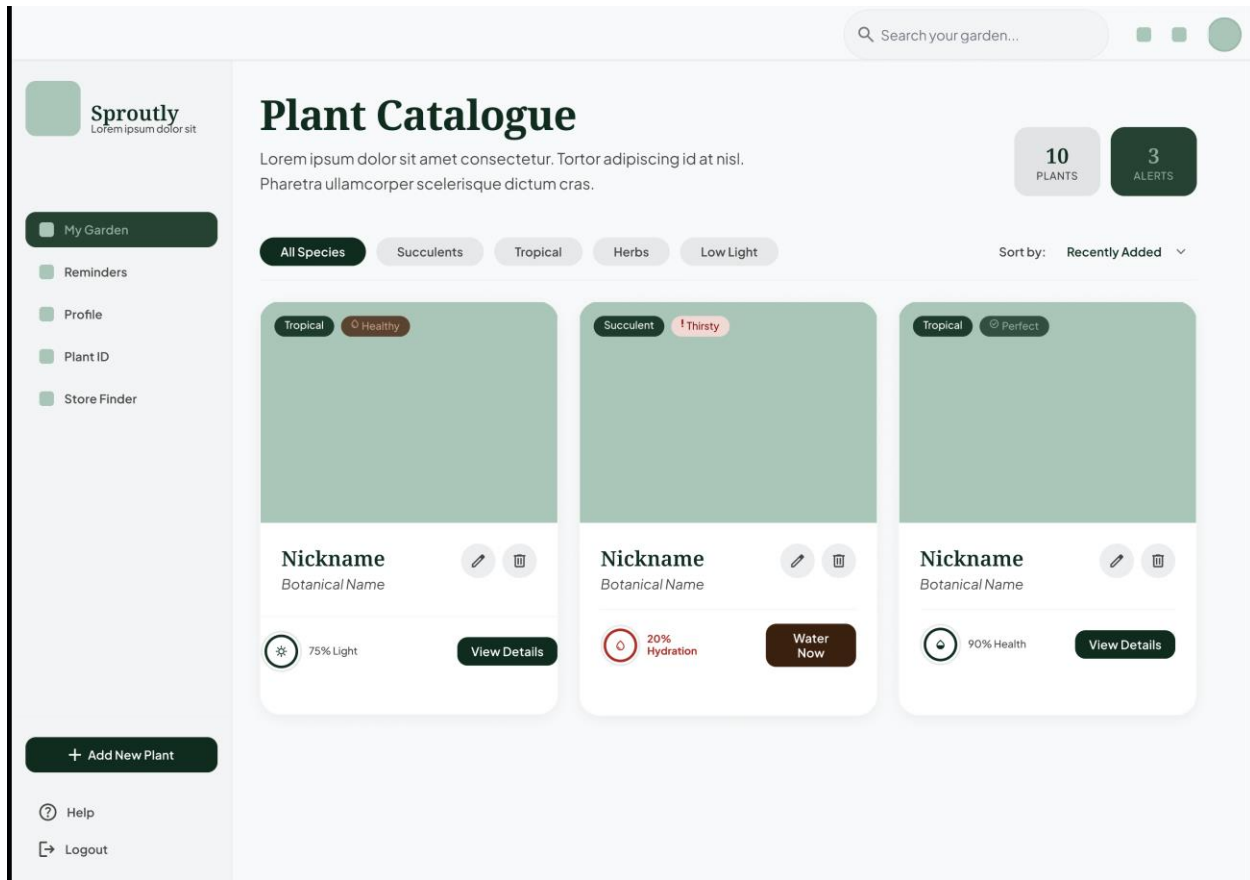
Dashboard – This screen displays what users can see once logged in

This page serves as an immediate plant status checker. Instead of the user going to the “My Garden” page, users can instantly see the high-level status of the plants



Plant Catalogue – This page displays the plants added by the user

In this page, we use cards to display the plants owned by the user. Users can immediately see the plant information like plant name, botanical name, and health status.



Add New Plant – Modal pop-up to add new plant to the user's catalogue

Add New Plant

×

Plant Portrait



Upload Photo

PNG, JPG up to 10MB

Plant Name

e.g., Aria

Species

Search species...

Placement

Select location

Acquisition Date

mm/dd/yyyy

Initial Notes (Optional)

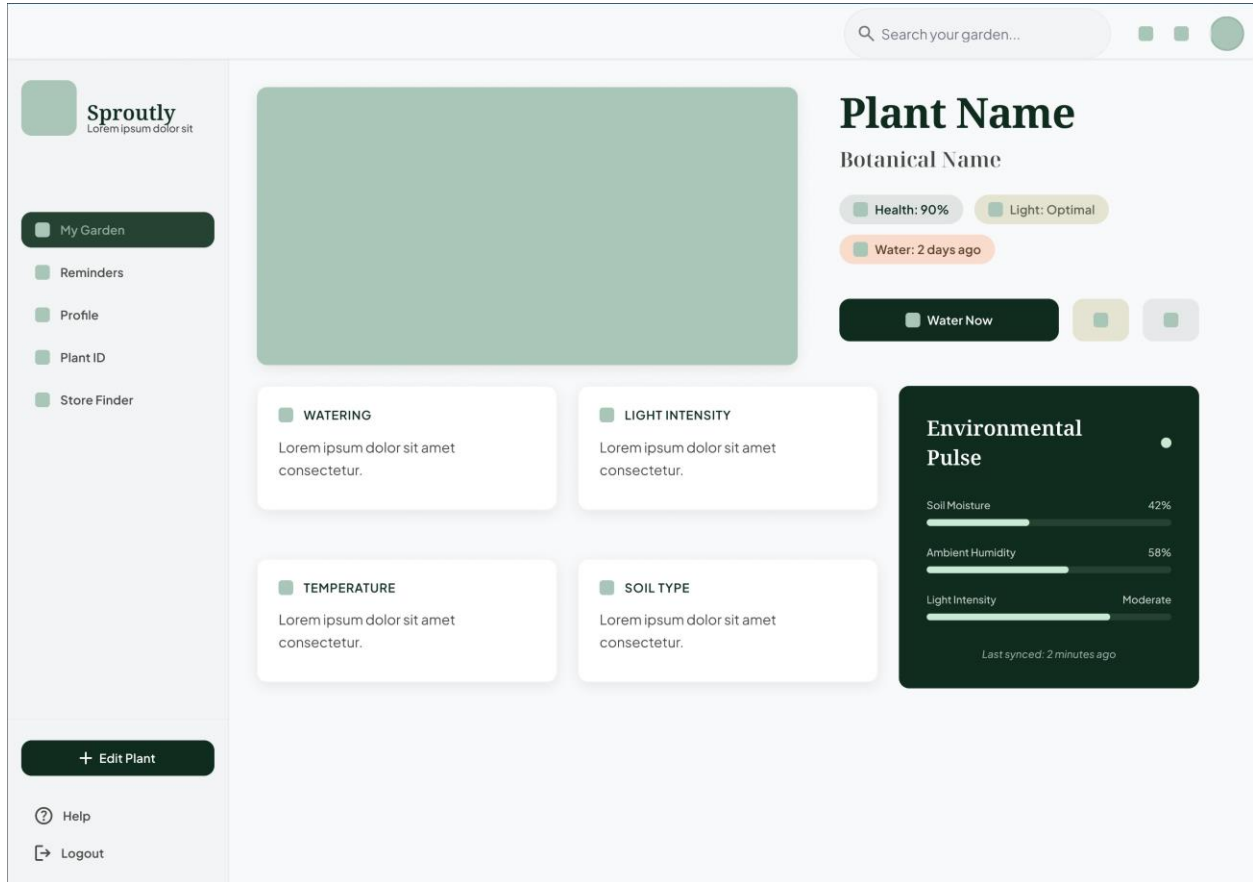
Describe the plant's health or specific needs...

Cancel

☐

Add to Garden

Plant Details – This page displays the plant information



Update/Delete Plant – Modal pop-up that allows the user to edit or delete plant

Update Plant

×

📷 Change Photo

Plant Name

Lorem ipsum

Species

Lorem ipsum

Placement

Lorem ipsum

▼

Acquisition Date

mm/dd/yyyy

Last Watered
12 Days Ago

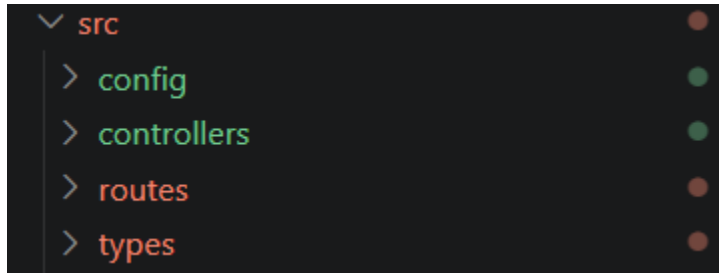
🗑 Delete

Cancel

Save Changes

Planned API routes/Backend

Configuring all our API routes in our SRC folder, containing config (database config), controllers (response handling), routes (holding express routes), types (contains all our model types). Of course, none of this is currently connected to our database, so many errors are showing up in our placeholder data, but it is a good demonstration of our route handling.



Config

Our MySQL database connection contains our database pool for now, for the purpose of pre-established cached connections.

```
src > config > TS database.ts > ...
1  // Database configuration file for MySQL
2  import mysql from 'mysql2/promise';
3  import dotenv from 'dotenv';
4
5  // Temporary placeholder database config
6  dotenv.config();
7
8  const pool = mysql.createPool({
9    host: process.env.DB_HOST,
10   user: process.env.DB_USER,
11   password: process.env.DB_PASSWORD,
12   database: process.env.DB_NAME,
13   waitForConnections: true,
14   connectionLimit: 10,
15   queueLimit: 0
16 });
```

Controllers

Our user controller, which handles all the CRUD operations pertaining to our user model, we've only added the GET request for now for demonstration purposes. We also will have a controller for our catalog, store finder, etc.

```
src > controllers > TS userControllers.ts > ...
1  // Temporary user controllers for demonstrations
2  import { Request, Response } from "express";
3
4  // Get user information
5  export const getUserInfo = (req: Request, res: Response) => {
6      const userId = req.params.userId;
7      // Temporary user data
8      const user = {
9          id: userId,
10         name: "John Doe",
11         email: "John@gmail.com",
12         password: "password123",
13     };
14     res.json(user);
15 }
```

Routes

These are all the routes for our current application. Each handles its own API calls for the user, e.g., get user notifications and add new user notifications.

```
src > routes > TS userRoutes.ts > ...
1 // Handles all user route API calls
2 const express = require("express");
3 const router = express.Router();
4
5 // Notification routes
6 // Temporary since it doesn't connect to the database yet, will be replaced in time with actual database content
7 let Notifications = [];
8
9 // Get all notifications for a user
10 router.get("/:userId", (req, res) => {
11   const userId = req.params.userId;
12   const userNotifications = Notifications.filter(
13     (notification) => notification.userId === userId,
14   );
15   res.json(userNotifications);
16 });
17
18 // Add a new notification for a user
19 router.post("/:userId", (req, res) => {
20   const userId = req.params.userId;
21   const { message } = req.body;
22   const newNotification = { id: Date.now(), userId, message };
23   Notifications.push(newNotification);
24   res.status(201).json(newNotification);
25 });
```

Types

These are most of our types, all relating to the user and heavily based on our ERD diagram.

```
15 // user type for user information (Making sure it follows our ERD design)
16 export interface User {
17   id: string;
18   name: string;
19   email: string;
20   password: string;
21 }
22
23 // store inventory type for the items in the store (Making sure it follows our ERD design)
24 export interface StoreInventory {
25   id: number;
26   name: string;
27   description: string;
28   price: number;
29   quantity: number;
30 }
31
32 // store details type for the store information (Making sure it follows our ERD design)
33 export interface StoreDetails {
34   id: number;
35   name: string;
36   description: string;
37   location: string;
38   contactInfo: string;
39 }
40
41 // Plant catalog type for the plants available in the store (Making sure it follows our ERD design)
42 export interface PlantCatalog {
43   id: number;
44   commonName: string;
45   scientificName: string;
46   description: string;
47   wateringGuide: string;
48   sunlightReq: string;
49   imageUrl: string;
50 }
51
```

API Endpoints

Plant Catalog Endpoints

```
src > routes > TS plantCatalogRoutes.ts > ...
1  // Plant catalog routes for managing plant information endpoints
2  import express, { Request, Response } from "express";
3  import {
4    getAllPlants,
5    getPlantById,
6    searchPlants,
7    filterByLightRequirements,
8    addPlant,
9    updatePlant,
10   deletePlant,
11 } from "../controllers/plantCatalogControllers";
12
13 const router = express.Router();
14
15 // GET/Get all plants in catalog
16 router.get("/", getAllPlants);
17
18 // GET/Search plants in catalog by name
19 router.get("/search", searchPlants);
20
21 // GET/Filter plants in catalog by sunlight requirements
22 router.get("/light", filterByLightRequirements);
23
24 // GET/Get plant by ID
25 router.get("/:id", getPlantById);
26
27 // POST/Add plant to catalog
28 router.post("/", addPlant);
29
30 // PUT/Update plant in catalog
31 router.put("/:id", updatePlant);
32
33 // DELETE/Remove plant from catalog
34 router.delete("/:id", deletePlant);
35
36 export default router;
```

Inventory Endpoints

```
src > routes > % storePlannerRoutes.ts > ...
1 // Store planner routes for managing store inventory and planning endpoints
2 import express, { Request, Response } from "express";
3 import {
4   getAllInventory,
5   getInventoryById,
6   getInventoryByCategory,
7   searchInventory,
8   getLowStockItems,
9   addInventoryItem,
10  updateInventoryItem,
11  updateQuantity,
12  deleteInventoryItem,
13  getStoreDetails,
14  updateStoreDetails,
15  getInventorySummary,
16 } from "../controllers/storePlannerControllers";
17
18 const router = express.Router();
19
20 // Inventory endpoints
21 // GET/Get all inventory items
22 router.get("/inventory", getAllInventory);
23
24 // GET/Search inventory items by name
25 router.get("/inventory/search", searchInventory);
26
27 // GET/Get filtered items
28 router.get("/inventory/category", getInventoryByCategory);
29
30 // GET/Get low stock inventory items
31 router.get("/inventory/low-stock", getLowStockItems);
32
33 // GET/Get specific inventory item by ID
34 router.get("/inventory/:id", getInventoryById);
35
36 // POST/Add new inventory item
37 router.post("/inventory", addInventoryItem);
38
39 // PUT/Update inventory item
40 router.put("/inventory/:id", updateInventoryItem);
41
42 // DELETE/Remove inventory item
43 router.delete("/inventory/:id", deleteInventoryItem);
44
45 // Store details endpoints
46 // GET/Get store information
47 router.get("/details", getStoreDetails);
48
49 // PUT/Update store information
50 router.put("/details", updateStoreDetails);
51
52 // Store planning endpoints
53 // GET/Get inventory summary
54 router.get("/summary", getInventorySummary);
55
56 export default router;
57
```

Both endpoints will be connected to a full-fledged controller for each page to properly handle all their CRUD operations in full.

Team Responsibilities

David Jaiyeola – Database Designer

Stryfe Vidmar – Backend Planner

Revienne Galang – Frontend Designer